CHAPTER 04: TEST ANALYSIS AND DESIGN

1. Core Syllabus Concepts & In-Depth Notes

4.1 Test Techniques Overview FL-4.1.1 (K2)

Test techniques support the tester during **Test Analysis ("what to test")** and **Test Design ("how to test")**. They provide a systematic method to create a relatively small but sufficient set of test cases, identify coverage items, and define precise test data (ISO/IEC/IEEE 29119-4).

Technique Category	Basis of Test Derivation	Core Strengths & Unique Characteristics
Black-Box (Specification-Based)	Specified behavior of the test object without reference to internal code structure.	Tests are independent of implementation; if code is refactored without spec changes, tests remain valid. Detects deviations from specification.
White-Box (Structure-Based)	Internal structure and processing (source code, architecture, data flows, workflows).	Takes entire implementation into account; created after design/code; enables objective code coverage measurement.
Experience-Based	Tester's knowledge, past experience, domain insight, and known developer error patterns.	Detects subtle, obscure defects often missed by black-box and white-box methods; highly complementary to formal techniques.

4.2 Black-Box Test Techniques

4.2.1 Equivalence Partitioning (EP) FL-4.2.1 (K3)

> **Core Principle:** Equivalence Partitioning divides input/output data into partitions based on the assumption that **all elements in a given partition are processed in the same way** by the system. If one value detects a defect, all other values in that partition are expected to detect the same defect. Therefore, *one test per partition is sufficient.*

- Partitions may be continuous or discrete, ordered or unordered, finite or infinite.
- Partitions must be **non-empty** and **must not overlap**.
- **Valid Partitions:** Values that should be processed by the test object according to specifications.
- **Invalid Partitions:** Values that should be ignored or rejected by the test object.

CRITICAL EXAM RULE — Isolation of Invalid Partitions (Avoid Defect Masking):
To prevent **defect masking** (a situation where one defect conceals another), **each invalid partition MUST be tested in isolation** (one invalid value per test case, combined only with valid values for other parameters). Valid partitions, by contrast, can be freely combined in a single test case.

EP Coverage (%) = Partitions Exercised by at Least One Test CaseTotal Number of Identified Partitions (Valid + Invalid) × 100%

Each Choice Coverage (Multiple Input Sets):
When a test item has multiple input parameters, **Each Choice coverage** requires that each partition from each input set is exercised at least once. It does not require testing all combinations of partitions, avoiding combinatorial explosion.

4.2.2 Boundary Value Analysis (BVA) FL-4.2.2 (K3)

Core Principle: BVA tests the boundaries of equivalence partitions. It is applicable **ONLY to ordered partitions** (numbers, sequential dates, chronological data). Developers make the most errors at boundaries (e.g., using `<` instead of `<=`, or off-by-one errors).

2-Value BVA (Craig 2002, Myers 2011)
For each boundary, there are **two coverage items**:
1. The **boundary value** itself.
2. Its **closest neighbor** belonging to the adjacent partition.

Example: Valid Integer Range [1 to 10]
- Min boundary (1): **1** (valid), **0** (invalid)
- Max boundary (10): **10** (valid), **11** (invalid)

Total test values: 0, 1, 10, 11 (4 tests)

3-Value BVA (Koomen 2006, O'Regan 2019)
For each boundary, there are **three coverage items**:
1. The **boundary value** itself.
2. **Both of its immediate neighbors** (left & right).

Example: Valid Integer Range [1 to 10]
- Lower boundary: **0, 1, 2**
- Upper boundary: **9, 10, 11**

Total test values: 0, 1, 2, 9, 10, 11 (6 tests)

Why is 3-Value BVA More Rigorous than 2-Value BVA?
If code is incorrectly written as `if (x == 10)` instead of `if (x <= 10)`, 2-value BVA test data ($x = 10, x = 11$) will **NOT** detect the flaw (both yield expected results for those inputs). However, 3-value BVA tests $x = 9$, which immediately triggers a failure!

BVA Coverage (%) = Number of Boundary Values (and Neighbors) ExercisedTotal Number of Identified Boundary Values (and Neighbors) × 100%

Visual Comparison of EP vs. 2-Value BVA vs. 3-Value BVA:

Technique	Test Values Derived for Range [1, 10]	Total Coverage Items
Equivalence Partitioning	Any valid (e.g. 5), Any invalid low (-3), Any invalid high (15)	3 partitions (100% EP coverage requires 3 test cases)
2-Value BVA	0 (invalid), 1 (valid), 10 (valid), 11 (invalid)	4 boundary values (100% 2-val coverage requires 4 test values)
3-Value BVA	0, 1, 2 (around lower) and 9, 10, 11 (around upper)	6 boundary values & neighbors (100% 3-val coverage = 6 values)

4.2.3 Decision Table Testing `FL-4.2.3 (K3)`

Core Principle: Decision tables test requirements that specify how **different combinations of conditions result in different outcomes**. They provide an exhaustive, systematic approach to document complex business rules, preventing logic gaps and contradictions.

Structure: Conditions (inputs/causes) and Actions (outputs/effects) form the **rows**. Each **column** corresponds to a **decision rule** defining a unique combination of conditions and associated actions.

Limited-Entry Decision Tables:
All conditions and actions are Boolean:
- `T` = Condition satisfied (True)
- `F` = Condition not satisfied (False)
- `–` = Condition is irrelevant for action outcome
- `N/A` = Condition combination is infeasible
- `X` = Action occurs | *Blank* = Action does not occur

Extended-Entry Decision Tables:
Conditions or actions take on multiple values, such as ranges of numbers, equivalence partitions, or discrete enumeration categories (e.g., Bronze, Silver, Gold).

Worked Example: E-Commerce Free Shipping & Discount Decision Table

Conditions & Actions	Rule 1	Rule 2	Rule 3	Rule 4
Condition 1: VIP Member?	T	T	F	F
Condition 2: Order > $100?	T	F	T	F
Action 1: Free Shipping	X	X	X	
Action 2: 10% Discount	X			

Decision Table Coverage (%) = Number of Feasible Columns (Rules) ExercisedTotal Number of Feasible Columns × 100%

Full vs. Minimized Decision Tables:
A full Boolean table with n conditions has 2^n columns. To save test effort, tables are **simplified** by removing infeasible columns, and **minimized** by merging columns where certain conditions do not affect the outcome (marked with a dash `-`).

4.2.4 State Transition Testing `FL-4.2.4 (K3)`

Core Principle: A **state diagram** models the behavior of a system by showing its possible states and valid state transitions. Transitions are instantaneous, triggered by an event, qualified by a guard condition, and may result in an action:

```
event [guard condition] / action
```

State Diagram vs. State Table: A state table lists current states in rows and events in columns. Table cells represent resulting transitions and actions. *Crucial distinction:* While state diagrams show only valid paths, a **state table explicitly highlights invalid transitions** (represented by empty cells).

Three Coverage Criteria for State Transition Testing:

Coverage Criterion	Coverage Items & Requirement for 100% Coverage	Rigor & Subsumption
1. All States Coverage	Coverage items are the **states**. Test cases must ensure that every single state is exercised at least once.	**Weakest criterion**; can be achieved without exercising all transitions.
2. Valid Transitions Coverage (0-Switch Coverage)	Coverage items are **single valid transitions**. Test cases must exercise every valid transition at least once.	**Most widely used** criterion. 100% valid transitions coverage *guarantees 100% all states coverage*!
3. All Transitions Coverage	Coverage items are **all transitions (valid and invalid)** shown in a state table. Tests all valid transitions AND attempts all invalid transitions.	**Most rigorous**. Mandatory for safety/mission-critical software. Guarantees both valid transitions and all states coverage!

AVOIDING DEFECT MASKING IN STATE TESTING:
When executing **invalid transitions**, test **only ONE invalid transition in a single test case**. If multiple invalid transitions are strung together, the first failure masks whether the subsequent invalid transition would have been handled properly!

Example: E-Commerce Order Lifecycle
States: [Cart] → [Placed] → [Shipped] → [Delivered] or [Cancelled].
- A test covering *Placed* → *Shipped* → *Delivered* exercises 3 valid transitions.
- Attempting *Delivered* → *Cancelled* is an invalid transition that must be rejected.

4.3 White-Box Test Techniques FL-4.3.1 TO FL-4.3.3 (K2)

White-box (structure-based) techniques derive tests from an analysis of the software's internal architecture, source code, data flows, and control logic. The syllabus focuses on two fundamental code-level techniques:

4.3.1 Statement Testing and Statement Coverage (FL-4.3.1)

Coverage Items: Executable code statements. The goal is to design test cases that execute statements until acceptable coverage is achieved.

Statement Coverage (%) = Number of Executable Statements ExercisedTotal Number of Executable Statements in Code × 100%

Limitation: 100% statement coverage ensures every statement executes at least once, but it does *not* ensure that all decision logic has been tested (e.g., missing an `else` branch with no statements). It also cannot detect data-dependent errors (e.g. division by zero occurring only when input is zero).

4.3.2 Branch Testing and Branch Coverage (FL-4.3.2)

Coverage Items: Branches. A **branch** is a transfer of control between two nodes in the control flow graph. Branches can be **unconditional** (straight-line sequential code) or **conditional** (decision outcomes: `True` / `False` from an `if` decision, switch cases, loop exits).

Branch Coverage (%) = Number of Branches ExercisedTotal Number of Branches in Code × 100%

THE SUBSUMPTION PRINCIPLE (CRITICAL EXAM FLASHPOINT):
Branch coverage subsumes statement coverage!
Any test suite that achieves **100% branch coverage** is mathematically guaranteed to achieve **100% statement coverage**. The reverse is **NOT TRUE**: 100% statement coverage does NOT guarantee 100% branch coverage (an `if` without an `else` only requires one test to execute the statement, leaving the False branch untested!).

4.3.3 The Value of White-Box Testing (FL-4.3.3)

Fundamental Strengths:

- Takes the entire implementation into account, uncovering defects even if specifications are vague, outdated, or incomplete.
- Provides an objective, quantifiable measurement of code coverage.
- Can be applied in static testing during dry runs of code, pseudocode reviews, and architecture control flow analysis.

Corresponding Weakness:

- **Cannot Detect Defects of Omission (Watson 1996):** If a business requirement was completely omitted from the code, white-box testing can achieve 100% coverage without ever detecting that the required feature is missing!

4.4 Experience-Based Test Techniques FL-4.4.1 TO FL-4.4.3 (K2)

Experience-based techniques leverage the tester's knowledge, intuition, domain expertise, and past failure insights. They are highly effective at finding edge-case defects overlooked by structured black-box and white-box methods:

4.4.1 Error Guessing (FL-4.4.1)

Anticipates the occurrence of errors, defects, and failures based on tester knowledge: how the software worked in the past, types of mistakes developers tend to make, and failures that occurred in similar applications.

- Errors/defects typically relate to: *inputs* (invalid characters, missing parameters), *outputs* (format errors), *logic* (missing conditions), *computation* (overflows), *interfaces* (mismatched data types), or *data* (uninitialized variables).
- **Fault Attacks:** A structured implementation of error guessing where testers compile lists of known vulnerabilities and design tests specifically to expose those faults (Whittaker 2002).

4.4.2 Exploratory Testing (FL-4.4.2)

Formal Definition: In **exploratory testing**, tests are simultaneously **designed, executed, and evaluated** while the tester actively learns about the test object.

Session-Based Testing Framework:

- Testing is conducted within an uninterrupted, defined **time box** (e.g., 60–90 mins).
- Guided by a **test charter** containing high-level test objectives.
- Findings and execution steps documented in **test session sheets**.
- Followed by a **debriefing** meeting between tester and stakeholders.

When is Exploratory Testing Best?

- When specifications are sparse, vague, or absent.
- Under severe time pressure to provide rapid quality feedback.
- As a powerful complement to formal techniques.
- Can incorporate other techniques (e.g. applying EP dynamically during exploration).

4.4.3 Checklist-Based Testing (FL-4.4.3)

Testers design and execute tests to cover test conditions from a structured checklist. Checklists are built from past experience, understanding of common failure modes, or quality heuristics (e.g., Nielsen's 10 usability heuristics).

- Checklist items are typically phrased as questions (e.g., *"Are all currency fields rounded to 2 decimal places?"*).
- **Maintenance:** Must be updated based on defect analysis. Remove obsolete items so checklists do not become too long (Gawande 2009).

- **Trade-Off:** Provides high coverage and guidelines in the absence of detailed test cases, but allows individual tester variability (higher coverage, but less repeatability).

4.5 Collaboration-Based Test Approaches FL-4.5.1 FL-4.5.2 FL-4.5.3

While traditional techniques focus on *defect detection*, collaboration-based approaches emphasize **defect avoidance** through shared communication and alignment across business, development, and testing.

4.5.1 Collaborative User Story Writing (FL-4.5.1 — K2)

The 3 C's of User Stories (Jeffries 2000):

- **Card:** Physical or digital medium describing the story.
- **Conversation:** Dialogue on how software will be used.
- **Confirmation:** The acceptance criteria verifying completeness.

The INVEST Quality Model (Wake 2003):

Independent • **N**egotiable • **V**aluable • **E**stimable • **S**mall • **T**estable. *(If a stakeholder cannot test a story, it lacks clarity!).*

Standard User Story Template: `As a [role], I want [goal], so that [business value]`, followed by Acceptance Criteria.

4.5.2 Acceptance Criteria (FL-4.5.2 — K2)

Acceptance criteria define the conditions an implementation must meet to be accepted by stakeholders. They serve as **test conditions**. Two primary formats:

Format	Structure & Definition	Concrete Syntax Example

Scenario-Oriented	Expresses behavior in natural language following the BDD Given/When/Then pattern.	`Given user has $50 balance`
		`When user transfers $20`
		`Then balance becomes $30`

Rule-Oriented	Verification checklist or tabulated input-output business rules.	• Passwords must be ≥ 8 characters. • Passwords must contain a number.

4.5.3 Acceptance Test-Driven Development (ATDD) FL-4.5.3 (K3)

Core Principle: ATDD is a collaborative test-first approach where test cases are created **prior to code implementation** from user stories and acceptance criteria (Adzic 2009).

The ATDD 3-Step Test Derivation Sequence:

1. **Positive Test Cases First:** Confirm correct behavior without exceptions (the "happy path" where everything goes as expected).
2. **Negative Test Cases Next:** Test error handling, exception flows, boundary rejections, and invalid inputs.
3. **Non-Functional Test Cases Finally:** Validate performance efficiency, usability, and security constraints.

Executable Requirements: When automated in tools like Cucumber or Robot Framework, acceptance test cases serve as living, executable specifications that validate the software upon every build.

2. High-Yield Exam Flashpoints & Key Distinctions

TOP 10 ISTQB CTFL v4.0.1 CHAPTER 4 EXAM TRAPS & DISTRACTORS:

1. **"100% statement coverage implies 100% branch coverage"** → **FALSE!** Branch coverage subsumes statement coverage, NOT vice versa. 100% statement coverage leaves unexecuted decision outcomes.
2. **"Invalid partitions should be combined in a single test case"** → **FALSE!** Invalid partitions must be tested *in isolation* to prevent defect masking. Valid partitions can be combined.
3. **"White-box testing finds defects of omission"** → **FALSE!** If a required feature is never written in code, white-box coverage can be 100% without detecting the missing requirement.
4. **"2-value BVA tests the boundary and both neighbors"** → **FALSE!** 2-value BVA tests the boundary and its *one* closest neighbor in the adjacent partition. 3-value BVA tests both neighbors.
5. **"State tables show only valid transitions"** → **FALSE!** State diagrams show valid transitions; state tables explicitly document *invalid transitions* as empty cells.
6. **"In state testing, test multiple invalid transitions in one test case"** → **FALSE!** Test only one invalid transition per test case to avoid defect masking.
7. **"ATDD derives negative test cases first"** → **FALSE!** ATDD derives *positive tests first*, followed by negative tests, and finally non-functional tests.
8. **"Error guessing is an ad-hoc unstructured technique with no methods"** → **FALSE!** Error guessing is implemented systematically through *fault attack lists*.
9. **"Exploratory testing has no documentation or structure"** → **FALSE!** Session-based exploratory testing uses time boxes, test charters, session sheets, and debriefings.
10. **"Use case testing is part of Foundation v4.0.1"** → **FALSE!** Use case testing was removed from Foundation Level v4.0 and moved to Advanced Test Analyst.

Rapid Comparison Matrix (Core Technique Distinctions)

Concept A	Concept B	Crucial Distinguishing Test
Statement Coverage	**Branch Coverage**	Branch coverage is strictly stronger and *subsumes* statement coverage. Statement tests lines; branch tests decision outcomes.
2-Value BVA	**3-Value BVA**	2-value tests 2 points per boundary; 3-value tests 3 points (boundary + left + right) and is more rigorous against relational faults.
State Diagram	**State Table**	Diagrams visualize valid flows; Tables explicitly capture *invalid transitions* in matrix cells.
Scenario-Oriented	**Rule-Oriented**	Scenario uses *Given/When/Then* (behavior-driven); Rule uses *bullet lists or condition tables*.
TDD	**ATDD**	TDD directs *unit code implementation*; ATDD directs *business feature acceptance from acceptance criteria*.

High-Yield Memory Mnemonics for Revision:

- **Subsumption:** *Branch > Statement* (Branch covers all statements; statement misses branches).
- **BVA Rule:** *2-Value = Boundary + 1 Neighbor | 3-Value = Boundary + 2 Neighbors*.
- **ATDD Order:** *Positive → Negative → Non-Functional*.
- **User Stories:** *3 C's = Card, Conversation, Confirmation | Quality = INVEST*.

3. Complete Chapter 4 Keyword Lexicon (Official Terms)

Acceptance Criteria The criteria that a component or system must satisfy in order to be accepted by a user, customer, or other authorized entity.

Acceptance Test-Driven Development (ATDD)

A collaborative test-first approach where team members define acceptance tests prior to coding to direct development.

Black-Box Test Technique A test technique based on an analysis of the specification of a component or system without reference to its internal structure.

Boundary Value Analysis (BVA) A black-box test technique based on exercising the boundary values of ordered equivalence partitions.

Branch A transfer of control between two nodes in a control flow graph (conditional decision outcome or unconditional jump).

Branch Coverage The coverage of branches in a control flow graph exercised by executed test cases, expressed as a percentage.

Checklist-Based Testing An experience-based test technique where the tester designs, implements, and executes tests to cover items from a checklist.

Collaboration-Based Test Approach An approach focusing on defect avoidance through cross-functional communication and shared authoring of stories and acceptance criteria.

Coverage The degree to which specified coverage items have been determined or exercised by a test suite, expressed as a percentage.

Coverage Item An attribute or entity that is defined as a basis for test coverage (e.g., an equivalence partition, a boundary value, or a branch).

Decision Table Testing A black-box test technique in which test cases are designed to exercise the combinations of conditions and resulting actions in a decision table.

Equivalence Partitioning (EP) A black-box test technique in which test inputs are partitioned into mutually exclusive groups assumed to be processed in the same way.

Error Guessing A test technique used to anticipate the occurrence of errors, defects, and failures based on the tester's knowledge and experience.

Experience-Based Test Technique A test technique based on the tester's experience, knowledge, and intuition to design and execute test cases.

Exploratory Testing An approach to testing where the testers dynamically design and execute tests based on their knowledge, exploration, and results of previous tests.

State Transition Testing A black-box test technique in which test cases are designed to exercise valid and invalid transitions between system states.

Statement Coverage The percentage of executable statements in source code that have been exercised by executed test cases.

Test Technique A procedure used to define test conditions, identify coverage items, and derive test cases systematically.

White-Box Test Technique A test technique based on an analysis of the internal structure, code, architecture, and control flow of the test object.

STUDENT NOTES & EXAM FLASHCARD REMINDERS (Handwritten Study Space)

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (Part 1)

Question 1 [K3 - FL-4.2.1]: An insurance application accepts an applicant's age as an integer input. Valid ages for coverage are from 18 to 65 inclusive. Ages under 18 are rejected with "Too young", and ages over 65 are rejected with "Too old". Inputs must be positive integers up to 120. Using Equivalence Partitioning, what is the MINIMUM number of test cases required to achieve 100% equivalence partition coverage?

A) 2 test cases

B) 3 test cases

C) 4 test cases

D) 6 test cases

Correct Answer: B
Rationale: The input range [1 to 120] is partitioned into: (1) Invalid partition: 1 to 17 ("Too young"), (2) Valid partition: 18 to 65 ("Accepted"), and (3) Invalid partition: 66 to 120 ("Too old"). To achieve 100% EP coverage, one test case from each of the 3 identified partitions is required: e.g., 10, 30, and 70 (Section 4.2.1).

Question 2 [K3 - FL-4.2.2]: For the age requirement above (valid range 18 to 65 inclusive), which set of test values achieves 100% coverage using 2-VALUE Boundary Value Analysis (BVA)?

A) 17, 18, 65, 66

B) 17, 18, 19, 64, 65, 66

C) 18, 65

D) 0, 18, 65, 120

Correct Answer: A
Rationale: In 2-value BVA, each boundary requires testing the boundary value and its closest neighbor in the adjacent partition. For the lower boundary (18): 18 (valid) and 17 (invalid). For the upper boundary (65): 65 (valid) and 66 (invalid). Thus, {17, 18, 65, 66} provides 100% 2-value BVA coverage (Section 4.2.2).

Question 3 [K3 - FL-4.2.2]: Which set of test values represents 3-VALUE Boundary Value Analysis for the lower boundary (18) of the valid range [18 to 65]?

A) 17 and 18

B) 17, 18, and 19

C) 16, 17, and 18

D) 18, 19, and 20

Correct Answer: B
Rationale: 3-value BVA requires exercising the boundary value itself and both of its immediate neighbors. For the boundary 18, the values are 17 (left neighbor), 18 (boundary value), and 19 (right neighbor) (Section 4.2.2).

Question 4 [K2 - FL-4.3.2]: A program contains a single `if...then` decision statement with no `else` block. A single test case executes the condition such that it evaluates to TRUE, running the inner code. What are the resulting Statement Coverage and Branch Coverage percentages?

A) 100% Statement Coverage, 100% Branch Coverage

B) 100% Statement Coverage, 50% Branch Coverage

C) 50% Statement Coverage, 50% Branch Coverage

D) 50% Statement Coverage, 100% Branch Coverage

Correct Answer: B
Rationale: All executable statements inside the `if` block were executed (100% Statement Coverage). However, an `if` decision has TWO branch outcomes: True and False. Since only the True branch was exercised, only 1 of 2 branches was covered (50% Branch Coverage). This demonstrates why 100% statement coverage does not imply 100% branch coverage (Section 4.3.2).

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (Part 2)

Question 5 [K3 - FL-4.2.4]: A digital banking account has three states: [Open], [Overdrawn], and [Suspended]. The system allows valid transitions: Open → Overdrawn, Overdrawn → Open, Overdrawn → Suspended, and Suspended → Open. What is the MINIMUM number of valid transitions that must be exercised to achieve 100% Valid Transitions Coverage (0-Switch)?

A) 3 transitions

B) 4 transitions

C) 6 transitions

D) 9 transitions

Correct Answer: B
Rationale: Valid transitions coverage (0-switch) requires exercising all single valid transitions. Since there are exactly 4 defined valid transitions (Open → Overdrawn, Overdrawn → Open, Overdrawn → Suspended, Suspended → Open), all 4 must be executed to achieve 100% valid transitions coverage (Section 4.2.4).

Question 6 [K2 - FL-4.3.3]: What is a primary limitation of white-box test techniques?

A) They cannot provide an objective measurement of code coverage.

B) They cannot detect defects of omission where requirements were never implemented in the code.

C) They cannot be applied during static code reviews or dry runs.

D) They are independent of how the software is designed or implemented.

Correct Answer: B
Rationale: Section 4.3.3 highlights Watson (1996): white-box testing derives tests exclusively from existing implementation; therefore, if a requirement was completely omitted by developers, white-box testing cannot detect the omission.

Question 7 [K2 - FL-4.4.2]: In session-based exploratory testing, what document provides the tester with high-level test objectives to guide their time-boxed exploration?

A) Detailed Test Procedure

B) Test Charter

C) Decision Table

D) Traceability Matrix

Correct Answer: B
Rationale: In session-based exploratory testing, the tester uses a **test charter** containing test objectives to guide the exploration within a defined time box (Section 4.4.2).

Question 8 [K3 - FL-4.5.3]: When using Acceptance Test-Driven Development (ATDD), in what order should the team derive and design test cases?

A) Non-functional tests → Negative tests → Positive tests

B) Negative tests → Positive tests → White-box branch tests

C) Positive tests → Negative tests → Non-functional tests

D) Exploratory tests → Fault attacks → Acceptance tests

Correct Answer: C
Rationale: Section 4.5.3 explicitly states that in ATDD: "Typically, the first test cases are positive... After the positive test cases are done, the team should perform negative testing. Finally, the team should cover non-functional quality characteristics."

Part 2: Additional Questions for Remaining Learning Objectives

Question 9 [K2 - FL-4.1.1]: A tester designs tests from personal experience of defects found in similar applications and knowledge of typical developer mistakes, without deriving the tests from a formal model or from the code structure. Which category of test techniques is this?

A) Black-box test techniques, because the tester does not look at the code.

B) White-box test techniques, because the tester uses knowledge of defects.

C) Experience-based test techniques.

D) Static test techniques.

Correct Answer: C

Rationale: Black-box techniques derive tests from specifications and models without referring to the internal structure. White-box techniques derive tests from the structure (code, architecture, detailed design). Experience-based techniques use the knowledge and experience of testers, developers, users and other stakeholders. Static techniques evaluate work products without executing code (Section 4.1).

Question 10 [K3 - FL-4.2.3]: A decision table for ATM cash withdrawal uses the conditions C1 "Card valid?", C2 "PIN correct?" and C3 "Sufficient balance?". Rules: R1 (C1=Y, C2=Y, C3=Y) dispense cash; R2 (Y, Y, N) show "insufficient funds"; R3 (Y, N, –) reject the PIN; R4 (N, –, –) retain the card. The "–" means the condition is irrelevant for that rule. What is the minimum number of test cases needed to achieve 100% decision table coverage?

A) 3

B) 4

C) 5

D) 8

Correct Answer: B

Rationale: Decision table coverage = number of rules (columns) exercised by at least one test case / total number of feasible rules. The table has 4 rules (the "–" values collapse the $2^3 = 8$ possible combinations into 4 rules), so one test per rule gives 100% coverage: **4 test cases** (Section 4.2.3).

Question 11 [K2 - FL-4.3.1]: Which statement about statement testing is correct?

A) 100% statement coverage means every executable statement was executed at least once, but a decision's False outcome may never have been exercised.

B) 100% statement coverage guarantees that every outcome of every decision was exercised.

C) Statement coverage is the percentage of test cases that executed without failure.

D) Statement testing can only be applied to specifications, not to source code.

Correct Answer: A

Rationale: Statement coverage = number of statements executed by the tests / total number of executable statements (as a percentage). With 100% statement coverage every executable statement has been executed at least once, but this does not guarantee that all decision outcomes were tested (that requires branch coverage) and it may not detect defects that depend on particular data (Section 4.3.1).

Question 12 [K2 - FL-4.4.1]: A tester prepares a list of likely faults from experience (division by zero, empty input, leap-year dates, null values) and designs tests to trigger them. Which technique is this?

A) Boundary value analysis, because the tests target typical limits.

B) State transition testing, because the tester models states.

C) Statement testing, because the tester examines the code.

D) Error guessing (using a fault attack), because the tester anticipates defects from experience and designs tests to expose them.

Correct Answer: D

Rationale: Error guessing is a technique in which testers anticipate errors, defects and failures based on their knowledge: how the application has worked in the past, the types of errors developers make, and failures seen in other applications. A systematic approach is the fault attack: list possible errors, defects and failures, then design tests that expose them (Section 4.4.1).

Question 13 [K2 - FL-4.4.3]: Which statement about checklist-based testing is TRUE?

A) Checklist items should be very general so that the checklist never needs updating.

B) A checklist can be derived from experience, from knowledge of what matters to users and from knowledge of how software fails; its items should be phrased so that they can be checked independently, often as questions.

C) The checklist must be completed once before testing starts and must never be changed.

D) Checklist-based testing provides precise, repeatable coverage metrics.

Correct Answer: B

Rationale: Checklists are built from experience, user priorities and understanding of why and how software fails. Good items are phrased as questions, are independently verifiable, do not overlap and are not too general; items that automated tools can check or that are better as entry/exit criteria should be excluded. Checklists should be updated regularly. Because checklists are high-level, repeatability and measurable coverage are limited (Section 4.4.3).

Question 14 [K2 - FL-4.5.1]: Which of the following is a well-formed user story?

A) "The system shall store passwords in a relational database."

B) "Implement the password reset service using SHA-256 during sprint 3."

C) "As a registered customer, I want to reset my password by e-mail so that I can regain access without calling support."

D) "Testers must manually verify the reset feature before every release."

Correct Answer: C

Rationale: A user story has three key elements: the role, the goal (functionality) and the business value. Good user stories follow INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) and the 3 Cs: Card, Conversation, Confirmation. Writing stories collaboratively with developers, business representatives and testers makes them testable (Section 4.5.1).

Question 15 [K2 - FL-4.5.2]: The acceptance criterion "Given a registered user with a valid password, When the user logs in, Then the dashboard is displayed" is written in which format?

A) Scenario-oriented acceptance criteria (Given/When/Then).

B) Rule-oriented acceptance criteria written as a bullet list of rules.

C) Rule-oriented acceptance criteria written as an input/output table.

D) It is not an acceptance criterion; it is a unit test case.

Correct Answer: A
Rationale: Acceptance criteria can be scenario-oriented (Given/When/Then, as used in BDD), rule-oriented (a bullet list of verification rules or an input/output table) or written in another format, as long as it is well defined and unambiguous. The example uses Given/When/Then, so it is scenario-oriented (Section 4.5.2).